

Group by Skeleton with Position - Data Importing on Large Spreadsheet

Mike Li

Abstract—Group by Skeleton with Position is a thought model and an associated implementation for data importing on large spreadsheet or structured data set with n-dimensions ($n \geq 2$). It is a general solution to import large spreadsheet. Users find the skeleton or primary keys or least common multiple or substitution (like $t = x + 1$) from the rows in spreadsheet, then group the rows by the skeleton. If the original data set contains 30 or more columns, then users find it can be grouped by only one column and its position (sheet number, row number, column number), that means the size of the key data for grouping reduced by almost 10 times. After group is done, then separate the group with only position and without skeleton into the sub procedure, and at the same time, the whole data set is also sent to the sub procedure. In the sub procedure, the system can get the required data according to the position and the whole data set. Due to the compressed size of the data set, and a few columns (generally 4) of group skeleton, the memory usage of them is most optimized in main procedure and sub procedure to avoid the memory lack issue.

Index Terms—data importing, group by skeleton with position, skeleton, large spreadsheet, substitution

I. INTRODUCTION

Over the past two years in SAP, there are two applications (Manage Prices – Sales, F4111; Manage Tax Rates – Sales, F6972) using the implementation to solve the large data set importing (can be half million or one million rows with 30 columns, depends on the row content). These 2 applications use specific method, but the similar thought to import large data set.

The original issue and question about the large data set of spreadsheets importing, is that although user can import the spreadsheet, but when the data is decompressed into memory, the issue occurs like out of memory or too large due to the huge size.

When all the data is split into small parts and throw them into sub procedures or execute them one by one sequentially, there will be no memory issue. So, the core problem is to group the spreadsheet without memory issue.

Generally, if the spreadsheet is 100 M, then the decompress size is 10 or more times of it. Group by skeleton (key figures), not group by the whole line, is the solution to avoid memory issue.

This work was supported in part by SAP SE. (Corresponding author: Mike Li).

Mike Li is with the SAP Labs China (Chengdu), Chengdu, China (e-mail: mike.li01@ sap.com).

II. USE CASE

When a project team with 20 members wants to finish a task, and the task contains 6 million rows in different spreadsheet, how can they finish the work in a restrict time based on spreadsheet?

If the uploading tool can process 30,000 rows, then team members need to separate the spreadsheet into 200 spreadsheets. When do the separation, data dependency and correction are considered, and the complexity is added.

If the uploading tool is improved to upload at least half million rows, then only separate 12 spreadsheet, 15 times reduced human work, and many human errors eliminated.

Current solution is to separate the half or one million rows by the automation tool without human work. The key is the separation rule

This is intended as an authoring template, not a final production template. It is not intended to match the final published format. Differences in final formatting are likely in the final IEEE files. Page count in the template is an estimate. Do not adjust line and character spacing to fit your paper to a specific length.

III. THOUGHT MODEL

The thought model is that for a given row with multiple columns find a skeleton (with position) to represent the row, then group by the skeletons and split them into small packages, at last parse the package by the compress source data set and the relevant positions.

A. Actions

- 1) Given large data set with compressed format (large size if decompress it).
- 2) Find key figures like finding the least common multiple (LCM) or substitution as skeleton (small size).
- 3) Group the data set by the skeleton to get the positions (small size).
- 4) Send the positions without key figures of each group and the compressed data set to sub procedure.
- 5) In sub procedure, data can be extracted according to the whole spreadsheet and give positions.

B. Notes

- 1) Each sub procedure is standalone from the business or requirement perspective.
- 2) If need the total count for action statuses, then when group the data, get the total count, in each sub procedure, get the separate counts according to the

action statuses.

C. Cases

- 1) No need to find key figures

If there is no key figure to find out, then just group the data with positions directly.

For example, if the spreadsheet contains 500,000 lines and each line is independent, group them by the positions and throw them into next procedure or sub procedures.

- 2) Find key figures to group rows

If the data set contains key figures like Document Number, then group the lines by key figures as skeleton, and at the same time, get its position (sheet no., row no., column no.).

After decompressing the spreadsheet sequentially line by line, all the groups are collected with the format <Key Figure 1, ..., Key Figure k, sheet no., row no., column no.>, for the Document example, it is like <Document Number, sheet no., row no., column no.>, then split the groups with same key figures into the same group.

After splitting, for each group, remove the key figures, only keep the positions, then throw the positions and the original spreadsheet into the next procedure or sub procedure.

D. Processing Methods

- 1) Sequentially process in the same procedure

If there is sequential requirement (for example, B must be after A) or no time limit, it is recommended.

- 2) Parallely process in different procedure

If it requires to process in less time, then process the data parallely.

IV. IMPEMETATION

For many programming languages, there are libraries to read the spreadsheet based on sequential stream and it is compressed in an acceptable size.

A. Implementation Steps

- 1) Read the sequential stream and get the shared string (can be postponed stepping 2, if memory is lack).
- 2) Parse the sheet content line by line, find the key figures as skeleton based on the shared string or its value itself, and its position {Skeleton, Sheet No., Row No., Column No.}.
- 3) Group the lines by the skeleton and its position, make sure the same skeleton in the same group.
- 4) For each group, according to the package size, collect the relative positions excluding the skeleton.
- 5) Pass the collection (positions) and the original stream to next procedure or sub procedure.
- 6) In the procedure, get the data of all columns according to the stream and positions, and do the rest operations like create, update, delete.

B. Execution Flow

- 1) Input Spreadsheet

TABLE I

Input Spreadsheet Sample Data

Key	Type	Unit	...	Column N
D1	S	1	...	test
D2	D	1	...	test
D3	D	1	...	test
D4	D	1	...	test
D5	S	1	...	test
D6	S	1	...	test
D1	S	1	...	test
D8	E	1	...	test
D2	D	1	...	test

- 2) Get skeleton and position

TABLE II

Pair of Skeleton and Position

Skeleton	Position
Key	Sheet No., Row No., Column No.
D1	1,2,1
D2	1,3,1
D3	1,4,1
D4	1,5,1
D5	1,6,1
D6	1,7,1
D1	1,8,1
D8	1,9,1
D2	1,10,1

- 3) Group by skeleton

TABLE III

Group by skeleton

Group	Skeleton	Position
Group 1	D1	1,2,1
	D2	1,3,1
	D3	1,4,1
	D4	1,5,1
	D1	1,8,1
	D2	1,10,1
Group 2	D5	1,6,1
	D6	1,7,1
	D8	1,9,1

- 4) Pass (Input, Position) to Sub Procedure

TABLE IV

Pass (Input, Position)

Procedure	Input Stream	Position
Procedure 1	Spreadsheet	1,2,1
		1,3,1
		1,4,1
		1,5,1
		1,8,1
		1,10,1
Procedure 2	Spreadsheet	1,6,1
		1,7,1
		1,9,1

- 5) Parse by input and positions

TABLE V

Parse					
Procedure	Key	Type	Unit	...	Column N
Procedure 1	D1	S	1	...	test
	D2	D	1	...	test
	D3	D	1	...	test
	D4	D	1	...	test
	D1	S	1	...	test
	D2	D	1	...	test
Procedure 2	D5	S	1	...	test
	D6	S	1	...	test
	D8	E	1	...	test

V. USAGE

The implementation can be done by most programming languages like SAP ABAP, C/C++, and so on.

It can be used by most use cases to import large spreadsheet.

VI. REFINEMENTS

If the key is sorted, then the group will improve quickly and use less memory; but it is difficult for data set with large size.

If the key figures contain more than one key, then merge all the keys into a skeleton. That means skeleton = (key 1|key 2|...|key k). Then compare skeletons and group the data by skeleton.

VII. EVALUATION ON PERFORMANCE

If a spreadsheet contains 500,000 rows and 30 columns, it can be imported without memory issue. Its time is more than 2 imports (one is 300,000 rows; another is 200,000 rows). The propriate rows are not the linear curve for the size. 1 million rows with 30 columns, or 2^{20} (1,048,576) rows with 30 columns can be grouped and split into different procedures without memory issue.

If the time is a less factor to consider, then use large size in one procedure or one main with many sub procedures. That may save the memory resource.

There are simple measurements about the importing size, like Total lines * Length of all columns, Lines of package size * length of all columns, Spreadsheet size, and so on. These factors can be a gross check for the upper limit size.

VIII. EVALUATION ON MEMORY

A. Ideal Case – Same Size for Each Column, One Column as Skeleton

Example: Document (Doc.) No. as skeleton, total 1 million Rows, N columns, each column with same size 20 characters.

TABLE VI

Case for Memory Evaluation

Doc. No.	C1	C2	...	CN
0000001	S1	D1	...	T11
0000002	S1	D3	...	T22
...	...	...	...	...
0999998	S6	D5	...	T88
0000001	S1	D1	...	T11
1000000	S8	D8	...	T00

The compressed spreadsheet as sequential stream is a must for all solutions. So, the difference is to group the data and split it without memory issue.

Measurement 1 - Original Group: by all columns

The memory cost can be like: Total Size = $1,000,000 * (N * 20)$

Measurement 2 - Group by Skeleton with Position: by Document No. and its position (Sheet No., Row No., Column No.)

The memory cost can be like: Total Size = $1,000,000 * (4 * 20)$

The ratio of the 2 approaches is $1,000,000 * (N * 20) / (1,000,000 * (4 * 20)) = N/4$.

If the N is 40, then the used memory by skeleton is reduced by 10 times.

B. Common Case - Different Size for Each Column, One Column or More Columns as Skeleton

The example still uses one column as skeleton like above.

Measurement 1 - Original Group: by all columns

The memory cost can be like: Total Size = $1,000,000 * (\text{Length of Column 1} + \dots + \text{Length of Column N})$

Measurement 2 - Group by Skeleton with Position: by Document No. and its position (Sheet No., Row No., Column No.)

The memory cost can be like: Total Size = $1,000,000 * (\text{Length of Skeleton} + \text{Length of Sheet No.} + \text{Length of Row No.} + \text{Length of Column No.})$

The ratio of the 2 approaches is $1,000,000 * (\text{Length of Column 1} + \dots + \text{Length of Column N}) / (1,000,000 * (\text{Length of Skeleton} + \text{Length of Sheet No.} + \text{Length of Row No.} + \text{Length of Column No.})) = (\text{Length of Column 1} + \dots + \text{Length of Column N}) / (\text{Length of Skeleton} + \text{Length of Sheet No.} + \text{Length of Row No.} + \text{Length of Column No.})$

If the N is 40, total length of all columns is 300, and total length of skeleton and position is $(10 + 1 + 7 + 2) = 20$, then the used memory by skeleton is reduced by $300 / 20 = 15$ times.

IX. RELATED WORKS

About the error check, it can be done by the application itself.

ACKNOWLEDGMENT

Thanks for Archer Zhang, Shaun Liu, Bright Long to design and Emma Tian, Richard Meng, Mingjiang Dai to develop the job program and test the uploading function in app Manage Prices – Sales.

REFERENCES

- [1]. S. G. Jeffrey Dean, "MapReduce: Simplified Data Processing on Large Clusters," USENIX Association OSDI, vol. 4, pp. 137-149, 2004.



Mike Li received the B.S. degree in mathematics and applied mathematics from Chongqing University, Chongqing, China, in 2008.

From 2008, he was an SAP engineer. From 2018 to 2026, he was a Senior Engineer in SAP Labs China, Chengdu.